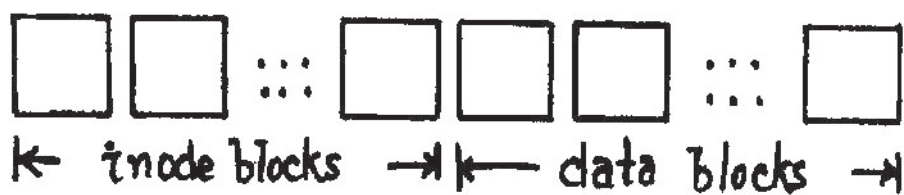
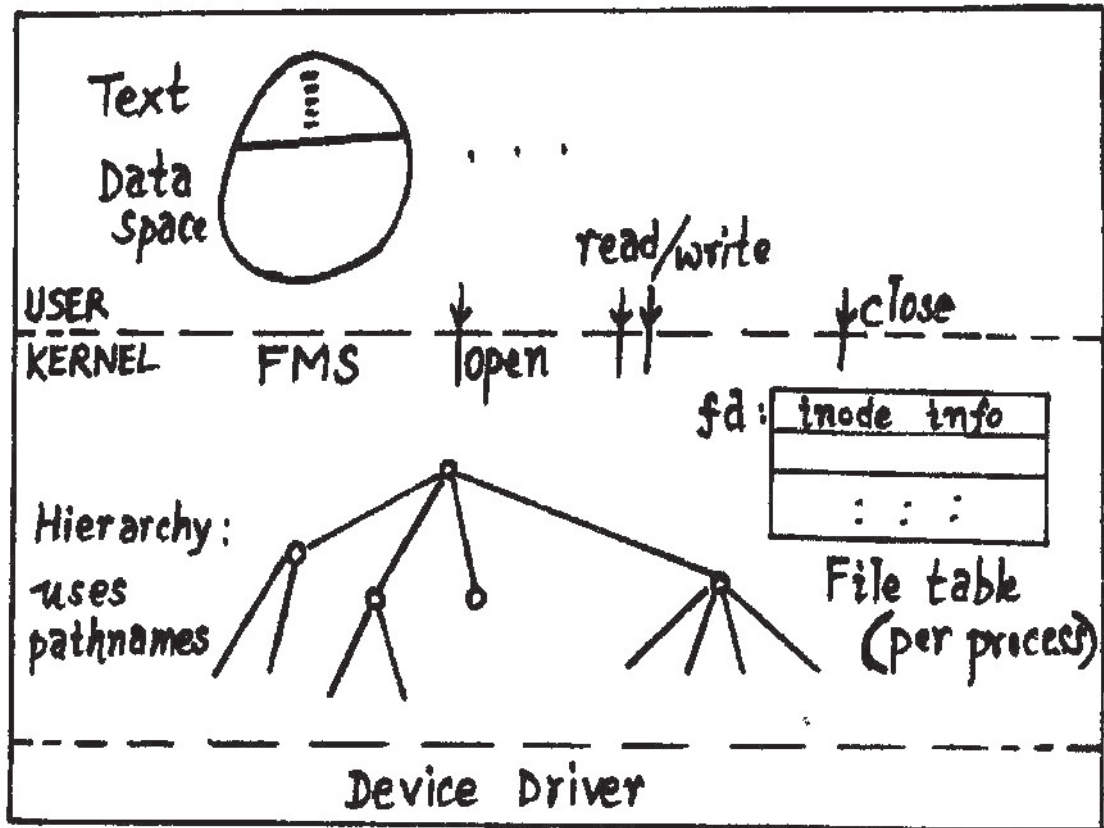


INTRODUCTION

on styles of usage

OVERALL PICTURE



Hard disk : Cylinders, Heads, Sectors

File Table: one for each process.

has NFILE rows : 0, 1, 2, ... NFILE - 1.

FMS : inode specialist

open system call

template: `int open(char *path, int flags, int perm);`

includes:

```
# include <fcntl.h>
# include <stdio.h>
# include <errno.h>
```

PATHNAME styles of specification
 absolute relative

Ex: `open("/usr/uttara/usens21/xxx/data/abc.dat", ...)`

Absolute pathnames : * very long
* non portable

current working directory : process is sitting
(read `pwd(1)`)

use relative pathname

Ex: `open("data/abc.dat", ...)`
`open("abc.dat", ...)`
`open("./abc.dat", ...)`

Pathname
{
 known a priori
 unknown a priori

pathnames

known a priori : if pathname changes,
recompilation needed

unknown a priori :

Sample : ledger.dat : very bulky,
unwieldy

split it : ledger1.dat , ledger2.dat, ... ledger12.dat

We want : single program for all months.

Sample Invocations

(a) known a priori :

```
.. = open ("abc.dat", ...);
```

(b) unknown a priori :

```
int month;
```

```
char path[15];
```

```
sprintf(path, "ledger%d.dat", month);
```

```
... = open (path, ...);
```

```
printf(3)
```

```
sprintf(3)
```

```
fprintf(3)
```

```
vprintf(3)
```

NOTE: We are computing pathname at exec. time.

(pathnames)

temporaries : need absolute pathnames

Need to be removed after the program finishes.

Create them in /tmp or /usr/tmp
(like BCC tubs)

Ex: /usr/tmp/temp : a natural name.

Catch: multitasking : can't have
duplicate names

(Duplicate names needed for IPC.)

Ex: /usr/tmp/zyx

/usr/tmp/2603

Catch: a process is an instantiation of
a program.

i.e. same program can be executed
multiple times.

hence: uniqueness cannot be based on the program
must be specific to a process.

NOTE: Each process has a body + soul

use: ps -e <CR> text+data pid number

pathnamestemporaries :

getpid (2) returns pid number

Ex: char path [40];
 sprintf (path, "/usr/tmp/temp%d",
 getpid ());
 ... = open (path, ...);

FLAGSRules: use | operator

use names: O_RDONLY, O_RDWR, O_WRONLY,
 O_SYNC, O_NDELAY,
 O_APPEND, O_TRUNC,
 O_CREAT, O_EXCL

PERMISSIONSneeded only if
O_CREAT is specified

chown (2) , chown (1) : change ownership

chmod (2) , chmod (1) : change permissions

permissions to start with

12 bits

sticky bit
set user-ID bit on exec
set group-ID bit on exec } only for some system executables

create: process needs write permission in the directory file.

9 bits: $\frac{rwx}{\downarrow}$ $\frac{rwx}{\downarrow}$ $\frac{rwx}{\downarrow}$
owner group others

universal practice: use octal digits

x permissions: usually zeros

group & others: usually same

write permissions: not given to others & group

owner: must have read & write

secret data: no write for group & others

Hence: 110 100 100 : 0644 4: Read
OR 110 000 000 : 0600 2: Write
OR 110100100B & 110000000B 1: Exec.

RETURN VALUE

on success: open returns row # of file table
called: file descriptor (fd)
OR : key, handle (token#)

Question: name to be used for descriptor.

Sample 1: only one file operated upon.

```
int fd;
fd = open(...);
read(fd, ...); write(fd, ...);
```

Sample 2: 2 files used.

```
int fd1, fd2;
fd1 = open("master.dat", ...);
fd2 = open("transact.dat", ...);
read(fd2, ...); /* read transaction */
write(fd1, ...); /* write master */
```

Sample 3: int master, transaction;

```
read(transaction, ...);
write(master, ...);
```

OPEN FAILURES

Invocation is a delegation of responsibility
open fails when

- (1) desired permissions not granted.
- (2) pathname component faulty (/usr/tmp..)
- (3) write permission in directory absent for create
- (4) file table full
- (5) disk read error, etc.

Actions : program stops.

open failure: open returns -1 .

(legal fd values : $\emptyset, 1, \dots, \text{NFILE}-1$).

System Calls

(a) return -1 on failure (if int is the return)

(b) return NULL on failure (if addr. is the return)

i.e. illegal looking returned value $\Rightarrow$ failure

reporting because of failure.

proper reports needed.

catch: caller may not be interested in the report.

ERROR INTERFACEcommon across
system callsopen always returns -1 on any failure

Report is in: extern int errno;

Error Symbolic Literals : listed in errno.h

E_ACCESS

E_IO

E_CHILD

E_PERMS

etc.

Analyse errno only after
a system call fails.open(2): lists error names.Sample :

```

int fd;
-----
fd = open (...);
-----
if (fd == -1) {
    switch (errno) {
        case E_PERMS :
            fprintf(stderr, "you cannot...");
            exit (1);
            -----
    }
}

```

NOTE: permission of
process is that
of user.**Messages**

regular error

exit(0): Success
exit(nonzero): Failure